

Notebook: GenAI

Angelika Merkel

2025-12-04

Table of contents

Preface

This is a notebook about my personal journey into generative AI, which I started during a break from work in the fall and winter of 2025. Here's the rough path I followed:

- **A short history of generative AI** – I started by using [ChatGPT](#) and [Perplexity](#) to get some background: how we got here, which milestones really mattered, and who the main players and models are today.
- **ChatGPT, the basics** – I subscribed to ChatGPT Pro (22 EUR/month) and worked through the beginner tutorials at the [OpenAI Academy](#) (free). This was quick but surprisingly solid to build a foundation and understand what the tool can do (and what not) and how to use the interface.
- **Prompt engineering** – To learn what I call *The art of talking to the machine*, I subscribed to [Coursera](#) (42 EUR/month) and took some basic and advanced courses on prompt engineering by Jules White, including how to create custom AI assistants (Custom GPTs) and build trustworthy genAI. This is where I started to get some real insights
- **Data analysis with ChatGPT** – Also by Jules White, these courses go deeper into practical workflows, showing how to use ChatGPT (via Code Interpreter) together with external tools for real data analysis tasks (and real magic).
- **Risks and Ethics of generative AI** - A compilation of insights and resources on how to provide appropriate rail guards for generative AI, AI agents and agentic AI and how (in my opinion) generative AI should be used ethically

1 Prompt Engineering

Based on my notes from the coursera courses '*Prompt Engineering with ChatGTP*', '*Advanced Prompt Engineering*', and '*Build Your CustomGPT*' by Jules White(Vanderbuilt University). Adapted with ChatGTP.

1.1 How LLMs Work (Conceptual + Technical Basics)

- **Large Language Models (LLMs)** generate text by predicting the *next token* based on patterns learned from training data.
 - They operate **autoregressively**: each generated token is appended to the context and used to predict the next one.
- **ChatGPT** = *Chat Generative Pre-trained Transformer*:
 - **Self-attention** computes weighted relationships across all tokens, allowing the model to focus on the most relevant parts of the context.
 - **Positional encodings** inject information about token order, enabling the model to represent sequences rather than bags of words.
- **Internal processing pipeline**:
 - Text → **tokenization** → **numerical embeddings** → stacked **transformer layers** → probability distribution over the next token.
- **Probability, not understanding**:
 - The model does not “know” facts; it estimates which continuation is most likely given the context and its training distribution.
- **Training objective**:
 - Minimize **cross-entropy loss**, pushing predicted token probabilities closer to the tokens observed in large-scale text corpora.
- **Determinism vs. randomness**:

- Sampling parameters (e.g. temperature, top-k/top-p) influence how deterministic or creative the output appears.

Example:

If you write “*The mitochondria is the ...*”, the model predicts “*powerhouse of the cell*” because this phrase has a high probability in the training data.

1.2 Prompt Engineering (Purpose and Practice)

“The art of talking to the machine.”

Prompt engineering aims to:

- Produce **reliable**, **repeatable**, and **useful** outputs despite inherent nondeterminism.
- Increase **predictability**, **consistency**, and **task alignment**.
- Clearly encode **task intent**, **context**, **constraints**, and **output structure**.
- Reduce ambiguity and lower the risk of hallucinations.

Example 1 (general):

Instead of:

> “Explain CRISPR.”

Better:

> “Act as a molecular biologist. Explain CRISPR to a motivated high-school student in 3 bullet points.”

Example 2 (technical):

Instead of:

> “Explain PCA.”

Better:

> “Provide a mathematically grounded explanation of PCA for an R programmer. Include eigenvectors, variance decomposition, and a 3-line R code example.”

2 Prompt Pattern (practical)

– A practical, example-driven overview.

2.1 Core Prompt Patterns

2.1.1 Persona Pattern

Helps control tone, expertise, cultural assumptions.

Example:

“Act as a senior bioinformatician experienced in single-cell epigenomics. Provide actionable suggestions.”

2.1.2 Audience Persona Pattern

Tell GPT *who you are* so explanations match your background.

Example:

“Assume I am new to machine learning but familiar with R and statistics. Explain what a transformer is.”

2.1.3 Question Refinement

GPT improves your question. Useful for technical tasks where question quality matters.

Example:

“When I ask you something unclear, suggest a sharper version first and ask if I want to use it.”

2.1.4 Few-shot Prompting

You show examples (introduce training data) → GPT follows your pattern. Useful for labeling, transformations, extraction tasks.

Example:

Input: ATGCGT

Output: reversed sequence: TGC GTA

Prompt:

Input: AACCAT

Output:

GPT continues with the intended pattern.

2.1.5 Flipped Interaction

GPT asks *you* the questions.

Example:

“Ask me clarifying questions until you can design a workshop outline. When you have enough information, present the final outline.”

2.1.6 Chain of Thought

Explicit reasoning steps.

Example:

“Show your reasoning in short bullet points and end with a boxed final answer.”

Helps when solving problems or planning.

2.1.7 ReAct Prompting

Reason → take external action → reason → answer.

Example:

“Search PubMed for recent papers on DNA methylation in HSCs, summarize top 3 findings, and explain how they relate to AML biology.”

(Assuming GPT has tools activated.)

2.1.8 Alternative Approaches

Ask for multiple solutions. (or strategy for approaches)

Example:

“Propose 5 different ways to create a metadata-capture workflow for a small lab, ranked by feasibility.”

Great for brainstorming and problem solving.

2.1.9 Evaluation Pattern

Use examples or rubrics to grade model output.

Example:

“Here are 3 example abstracts. Score the following abstract for clarity (1–5), structure (1–5), scientific accuracy (1–5). Provide justification.”

2.1.10 Game-play Pattern

Turn tasks into structured exercises. Define rules.

Example:

“We are going to play a game:

I describe a lab workflow, you generate 5 mistakes a junior researcher might make, and how to fix them.”

2.1.11 Recipe Pattern

Stepwise instructions with gaps.

Example:

“Fill in missing steps for this workflow:

- 1.
2. Load FASTQ files
- 3.
4. Align using BWA
- 5.
6. Generate BAM summary”
- 7.

2.1.12 Template Pattern

Use placeholders → consistent output.

Example:

Mutation Summary

Gene: <GENE>

Variant: <VARIANT>

Impact: <IMPACT>

Evidence: <QUOTES FROM ARTICLE>

2.2 Text Manipulation Patterns

2.2.1 Semantic Filter

Remove or replace specific elements.

Example:

“Remove all dates from the text but change as little as possible.”

2.2.2 Meta-language Pattern

Define a micro-language the LLM should learn.

Example:

“When I write ‘QC-S1’, interpret it as: run initial QC on sequencing files and summarize FastQC results.”

2.3 Verifier Pattern

2.3.1 Cognitive Verifier Pattern

Meta-reasoning pattern where the model is instructed to generate an answer, independently verify, critique, or test that answer and revise if needed. Focuses on error detection, not explanation.

Typical prompt form:

“First provide an answer. Then verify it by checking assumptions, edge cases, and alternative interpretations.” “Critically review your own solution and correct any mistakes.”

What it does well - Reduces hallucinations - Improves factual and logical reliability - Surfaces hidden assumptions - Excellent for high-stakes outputs

What it is not - Not primarily about explaining reasoning to the user - Verification may be concise or structured rather than narrative

Example:

Answer: X

Verification: - Check assumptions → valid - Alternative method → same result - Edge cases → none violated

Conclusion: Answer stands.

2.3.2 Fact Checking Pattern

Force verification.

Steps:

2. Extract facts
3. Compare with source
4. Flag inconsistencies

5. Insert verified fact list at the end

Example:

“Extract all factual statements from the text and list any that need verification.”

2.4 Workflows

2.4.1 Working on smaller pieces

Outline Expansion Start small, expand in pieces.

Example:

“Here is the outline for a workshop. Expand only section 2. Keep the rest unchanged.”

2.4.2 Tail Generation

Add automatic “post-processing.”

Example:

“When done summarizing, also produce a 3-item action list.”

3 Prompt Pattern (technical)

Technical Summary (with Examples)

A compact, engineering-oriented overview

3.1 Core Prompt Pattern

3.1.1 Persona Pattern (Functional Control)

Define computational role + constraints.

Example:

“Act as a senior data engineer. Output only SQL, no explanations.”

3.1.2 Audience Persona Pattern (Adjust Explanation Level)

Tells the model who the target reader is.

Example:

“Explain transformer attention to someone familiar with matrix multiplication but new to NLP.”

3.1.3 Flipped Interaction Pattern (Information Acquisition)

Useful when task requires parameters or architecture definition.

Example:

“Ask me all required questions to define a Snakemake workflow. Then generate the final workflow.”

3.1.4 Question Refinement Pattern (Error Reduction)

GPT rewrites your question to reduce ambiguity.

Example:

“Before answering, propose a more precise version of my question optimized for computational reproducibility.”

3.1.5 Cognitive Verifier Pattern (Structured Reasoning)

Model decomposes problem → produces final synthesized answer.

Useful to check the model’s reasoning. Improves the output.

Example:

“Break the question into technical sub-questions (5), answer them, then produce the final integrated explanation.”

3.1.6 Few-Shot Pattern (Pattern Imitation)

Provide input/output examples → model induces rule.

Example:

Input: “ATGGCT”

Output: reverse_complement: “AGCCAT”

Input: “CGTAAT”

Output:

GPT generalizes rule across new inputs.

3.1.7 Chain-of-Thought (CoT) Pattern (Explain Reasoning)

Model outputs intermediate reasoning (optional).

Example:

“Show your reasoning in short bullet points and end with a boxed final answer.”